Roll-a-Ball Game: Assignment 1 Report

General information

Item	Details
Game title	Roll a Ball
Demo video	https://drive.google.com/file/d/1c_WwAN-qQ5MPdVapSjEdj8-1ojTZGcPi/view?usp=drive_link
Unity project files	https://drive.google.com/file/d/1WwRvEWtxpxHkheoiefErFooC15Fb9fyI/view?usp=drive_link
Unity version	Unity 6.3 LTS (6000.3.25f1), URP

How to run the game

Option 1: play the build (macOS)

1. Download the build zip from the link above and double-click it to unzip.
2. Right-click **Roll-a-ball** and then double click to open.
3. Click **Play** on the main menu.

Option 2: open the Unity project

1. Download the project zip and unzip it.
2. In Unity Hub, click **Add → Add project from disk**, select the folder, and open it with Unity 6.3 LTS.
3. Open **Assets → Scenes → MainMenu** and press **Play**.

Overview

I built the Roll-a-Ball game from the Unity Learn tutorial and extended it with custom textures, a custom Shader Graph shader, and a new user interface.

- **Engine:** Unity 6.3 LTS (6000.3.25f1), Universal Render Pipeline (URP), macOS
- **Scenes:** MainMenu (start screen) and Minigame (the playable level)
- **Goal of the game:** Roll the ball to collect all 12 yellow pickups while avoiding a red enemy that chases the player

How to play

- **Controls:** Arrow keys to roll the ball; Space to pause and resume.

- **Objective:** Collect all 12 golden treasure cubes scattered around the arena.
- **Enemy:** A red block chases the ball/player around the obstacles.
- **Win:** Collect all 12 treasures before the enemy catches you.
- **Lose:** The enemy touches the ball/player before all treasures are collected.
- **Movable obstacles:** some light boxes can be pushed out of the way with the ball.

Core game (tutorial)

The complete tutorial is implemented: movement, pickups, scoring, an enemy, and win and lose conditions.

- **Player:** a sphere with a **Rigidbody** and **Sphere Collider**. A Player Input component sends movement from the **Input System** (arrow keys) to the **PlayerController** script, which applies force with **Rigidbody. AddForce**, so the ball rolls with physics.
- **Camera:** the **CameraController** script keeps a fixed offset from the ball in **LateUpdate**, so the camera follows smoothly after physics has moved the ball.
- **Level:** a textured ground plane surrounded by walls, plus static obstacles (blocks and pillars) and light movable boxes. The movable boxes have a **Rigidbody** with a small mass, so the ball can push them; the fixed obstacles have no Rigidbody and stay in place.
- **Pickups:** one cube was made into a **prefab** and placed 12 times. Each has a trigger collider and the "PickUp" tag, and the Rotator script spins it every frame using Time.deltaTime. On contact, OnTriggerEnter hides the pickup and increases the count.
- **Enemy:** a **NavMesh** Surface was baked on the ground, and the enemy uses a NavMesh Agent. The EnemyMovement script sets the player's position as its destination every frame, so it finds a path around obstacles.
- **Win and lose:** collecting all pickups shows "You Won!" and removes the enemy. If the enemy touches the ball, **OnCollisionEnter** detects the "Enemy" tag, destroys the ball, and shows a red "You Lost!" message.

Textures (extra feature)

I replaced the tutorial's plain colors with image textures on the level and the ball, and the ball changes its texture during play.

- **Level:** I imported brick, wood and concrete images into a Texture folder and created URP Lit materials with them as the Base Map. The floor uses a brick floor texture, the walls and blocks use brick textures, and the round pillars use a red wood texture.
- **Tiling:** on large surfaces such as the floor and long walls, I increased the material's Tiling values so the image repeats at a natural size instead of stretching and blurring.
- **Ball textures:** the PlayerController has a public Texture[] ballTextures list filled in the Inspector. At start and after every pickup, ChangeBallTexture() sets the ball material's main texture to the next image, using count % ballTextures.Length so it loops through the list.

Shaders (extra feature)

I created a custom **URP Lit shader** in Shader Graph, called BallRimGlow, that keeps the ball's texture and adds a glowing outline around its edges.

1. **MainTex** (Texture2D property, reference `_MainTex`, set as the main texture) feeds a **Sample Texture 2D** node, whose RGBA output goes to **Base Color**. Because it is the main texture, the ball-texture script can still swap images at runtime.
2. A **Fresnel Effect** node (Power 3) produces a value that is close to 0 in the center of the sphere and close to 1 at its edges, where the surface faces away from the camera.
3. A **Multiply** node combines the Fresnel value with **RimColor**, an HDR color property, and the result goes to **Emission**. Only the edges therefore emit light, which creates the rim glow.

I created a material (BallRimMat) from this shader, assigned a texture and a rim color in the Inspector, and applied it to the player ball.

New interface (extra feature)

I added a main menu scene, a pause menu, and clearer in-game messages, all built with Unity UI and TextMeshPro.

- **Main menu scene:** a full-screen background image, a "Roll a ball" title with a color gradient and outline, and Play and Quit buttons. The MainMenu script loads the Minigame scene with `SceneManager.LoadScene`; Quit closes the game, or stops Play mode in the editor. Both scenes are in the Build Profiles scene list, with MainMenu first.
- **Progress counter:** the count shows progress.
- **Win and lose messages:** the same text object shows "You Won!" or a red "You Lost!", with the color set in code. A Play Again button appears when the game ends.
- **Pause menu:** pressing Space calls the GameManager script, which sets `Time.timeScale` to 0 to freeze physics, the enemy and the rotating pickups, and shows a panel with Resume, Restart and Main Menu buttons. Pressing Space again resumes. Pausing is disabled after a win or loss.

All buttons are connected through their On Click () events to public functions in the GameManager and MainMenu scripts.

Scripts

The game uses six C# scripts.

Script	Attached to	Purpose
PlayerController	Player	Physics movement, pickups, counter, ball textures, win and lose
CameraController	Main Camera	Follows the ball at a fixed

Script	Attached to	Purpose
		offset
Rotator	Pickup prefab	Spins the pickups
EnemyMovement	Enemy	Chases the player with a NavMesh Agent
GameManager	Canvas (Minigame)	Pause, resume, restart, return to menu
MainMenu	Canvas (MainMenu)	Play and Quit buttons

Challenges and conclusion

A few problems taught me how Unity's parts depend on each other:

- **Script not running:** the file name and class name did not match, so Unity could not use the script. Renaming the file fixed it.
- **Ball not moving:** the Player Input component had no default action map selected, so movement messages were never sent.
- **Enemy in the wrong place:** the visible enemy body was offset from its NavMesh Agent parent, so it appeared to go the wrong way. Resetting the child's local position fixed it.
- **Camera error after losing:** the camera kept following the destroyed ball, so I added a null check.
- **Changes disappearing:** edits made in Play mode are reset when Play stops, so all setup must be done outside Play mode.

The finished game meets the tutorial requirements and adds textured surfaces, a ball whose texture changes during play, a custom rim-glow shader, a main menu, and a pause menu.